VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI- 590 018

"AICTE Activity Point Programme"

Submitted in Partial Fulfillment for the Award of Degree of

Bachelor of Engineering
in
Information Science And Engineering

Submitted by

Rakshith H N | 1MP23IS043

Department AICTE Activity Faculty Incharge

MS.Maheswari

Assossiate Professor

BGS COLLEGE OF ENGINEERING AND TECHNOLOGY
CA Site No. 6 & 7,3rd Main, Pipeline Road, Chord Rd, adj. to Mahalakshmi Metro Station, 2nd Phase, Stage 2, Mahalakshmipuram, Bengaluru, Karnataka 56008

BGS COLLEGE OF ENGINEERING AND TECHNOLOGY
CA Site No. 6 & 7,3rd Main, Pipeline Road, Chord Rd, adj. to Mahalakshmi Metro Station, 2nd Phase, Stage 2, Mahalakshmipuram, Bengaluru, Karnataka 560086

TABLE OF CONTENTS

S.No.	Title	Page No.
—	Table of Contents	**i**
—	List of Tables	**ii**
—	Abstract	**iii**
1.	**Introduction**	**1**
	1.1 Background	**1**
	1.2 Motivation	**2**
	1.3 Objectives	**2**
	1.4 Scope of the Project	**2**
2.	**Problem Statement**	**3**
3.	**Proposed System**	**4**
	3.1 System Overview	**4**
	3.2 Technologies Used	**4**
	3.3 System Architecture	**5**
	3.4 Algorithm — K-Means++	**6**
	3.5 OpenMP Parallelization	**7**
	3.6 Visualization Components	**7**
4.	**Result Analysis**	**9**
	4.1 Performance Evaluation	**9**
	4.2 Effect of K on Output Quality	**9**
	4.3 Comparison: Sequential vs. Parallel	**10**
	4.4 Limitations	**10**
5.	**Conclusion and Future Work**	**11**
6.	**References**	**12**

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Project File Structure	**5**
Table 3.2	Technologies and Tools Used	**5**
Table 3.3	Keyboard Controls Reference	**8**
Table 3.4	Time Complexity Analysis	**8**
Table 4.1	Effect of K on Performance and Quality	**9**
Table 4.2	Sequential vs. Parallel Execution Times	**10**

ABSTRACT

This report presents the design and implementation of a real-time color quantization system that applies the K-Means++ unsupervised clustering algorithm to live webcam video frames. The system is built entirely in the C programming language using the Simple DirectMedia Layer 3 (SDL3) multimedia framework and leverages OpenMP for CPU-level multi-threading to achieve real-time performance. Each incoming video frame is processed independently: pixels are grouped into K color clusters, and all pixel colors are replaced with their corresponding cluster centroid for visual display.

The system provides three simultaneous visual outputs: a color-quantized live camera feed, a luminance-sorted color palette bar, and an interactive 3D rotating RGB color cube showing centroid positions. The number of clusters K can be adjusted interactively at runtime. The combination of K-Means++ initialization (which reduces convergence iterations by 4–6 times compared to random initialization) and OpenMP parallelism (which distributes

CHAPTER 1: INTRODUCTION

1.1 Background

In digital image representation, colors are encoded as combinations of Red, Green, and Blue (RGB) channel values. A standard 24-bit RGB image can represent over 16.7 million distinct colors. While this allows for photorealistic imagery, many practical applications require a reduced color palette — either due to hardware constraints, storage limitations, artistic requirements, or computational efficiency.

Color quantization is the process of mapping the large color space of an image down to a smaller set of representative colors, called a palette, while minimizing perceptual distortion. Applications include GIF image encoding (limited to 256 colors), embedded display systems, color-palette based game art, texture compression, and image segmentation.

The K-Means algorithm is one of the most widely studied and applied unsupervised machine learning algorithms. Given a set of data points and an integer K, it partitions the data into K clusters such that the within-cluster sum of squared Euclidean distances (inertia) is minimized. When applied to the RGB pixel data of an image, K-Means naturally finds K representative colors that best describe the image's color distribution.

1.2 Motivation

While K-Means color quantization has been extensively studied for static images, its application to real-time video streams presents a significant computational challenge. Existing implementations are designed for offline batch processing, where time constraints are not critical. The motivation for this project is to explore whether the combination of the improved K-Means++ initialization strategy and CPU-level multi-threading via OpenMP can make per-frame K-Means clustering feasible in a real-time video pipeline.

Furthermore, the visual nature of color quantization provides an excellent opportunity to build an interactive educational tool that helps users understand how K-Means clustering works by observing its effect live on camera input.

1.3 Objectives

- To implement the K-Means++ clustering algorithm in C for color quantization of RGB image data.
- To parallelize the K-Means computation using OpenMP to achieve real-

CHAPTER 2: PROBLEM STATEMENT

Standard color quantization algorithms are designed for offline, batch-mode processing of static images. Applying these techniques to real-time video streams introduces a set of engineering challenges that must be addressed to maintain acceptable frame rates:

2.1 Computational Cost Per Frame

A standard webcam frame at 420×240 resolution contains approximately 100,800 pixels, each represented as a 3-dimensional point in RGB space. For K clusters, the K-Means assignment step requires computing K Euclidean distances for each of the N pixels, yielding $O(N \times K \times D)$ floating-point operations per iteration, where $D = 3$ for RGB. For $K = 8$ and 10 iterations, this amounts to approximately 24 million floating-point operations per frame. At 30 FPS, the system must perform approximately 720 million operations per second from the clustering step alone, making sequential execution insufficient on most consumer CPUs.

2.2 Poor Convergence with Random Initialization

The classical K-Means algorithm initializes cluster centroids randomly from the data. This frequently results in poorly separated initial centroids, causing the algorithm to require many iterations (often 15–30) before convergence. In a real-time system, a higher number of iterations per frame directly reduces achievable frame rate, as each iteration must complete before the next frame can be acquired and rendered. Additionally, poorly initialized centroids can lead to empty clusters, causing unstable quantization results that flicker visibly between frames.

2.3 Real-Time Rendering and Interactivity

Beyond the clustering computation, the system must also handle frame acquisition from the camera, pixel-format conversion, texture upload to the GPU, 3D visualization rendering, event handling, and final frame presentation — all within the same rendering iteration. Coordinating these operations without introducing visual artifacts (tearing, stuttering, or lag) requires careful design of the rendering pipeline.

2.4 Summary

In summary, the core problem addressed by this project is:

CHAPTER 3: PROPOSED SYSTEM

3.1 System Overview

The proposed system is a real-time color quantization application written in C that uses SDL3 for camera access and rendering, and OpenMP for parallel computation. It captures live frames from a connected webcam, applies K-Means++ clustering to each frame independently, replaces pixel colors with their cluster centroid values, and presents the result alongside a palette bar and 3D RGB visualization.

3.2 Technologies Used

Table 3.2 lists the primary technologies and tools used in the implementation.

Component	Technology / Tool	Version / Details
Programming Language	C (C11 Standard)	GCC via w64devkit
Graphics & Multimedia	SDL3 (Simple DirectMedia Layer)	SDL3.dll
Text Rendering	SDL3_ttf	SDL3_ttf.dll
Camera API	SDL3 Camera Subsystem	SDL_camera.h
Parallelism	OpenMP	-fopenmp flag (GCC)
Math Library	libmath (C standard)	-lm (math.h)
Build System	GNU Make	Makefile
Optimization	GCC -O3 flag	Aggressive optimization

Table 3.2: Technologies and Tools Used

The project file structure is shown in Table 3.1.

File	Purpose
kmeans.c	Main application: SDL3 initialization, render loop, 3D visualization, event handling
kmeans.h	K-Means++ engine with OpenMP parallelism (header-only

3.3 System Architecture

The application uses SDL3's callback-based main loop architecture, defined by three primary callbacks registered with the SDL3 runtime:

- **SDL_AppInit():** Called once at startup. Initializes the window (1280×720), renderer, camera device, OpenType font (Arial 50pt), and allocates memory for the centroid array. Opens the first available camera device using `SDL_OpenCamera()`.
- **SDL_AppIterate():** Called every frame. This is the main processing loop — it acquires the camera frame, runs K-Means++, updates pixel colors, uploads the texture, and renders all three visual components.
- **SDL_AppEvent():** Called on user input events. Handles keyboard events for adjusting K, viewport size, and 3D rotation angle.
- **SDL_AppQuit():** Called on exit. Frees all allocated memory and closes the camera device.

The data flow within each frame iteration is described below:

1. **Frame Acquisition:** `SDL_AcquireCameraFrame()` retrieves the latest raw frame from the camera hardware.
2. **Format Conversion:** `SDL_ConvertSurface()` normalizes the frame to packed RGB24 format (3 bytes per pixel: R, G, B).
3. **Scaling:** `SDL_ScaleSurface()` resizes the frame to match the current camera viewport dimensions.
4. **K-Means Clustering:** `kmeans2()` is called with the flat pixel array. Returns cluster assignments for all pixels and final centroid values.
5. **Pixel Replacement:** Each pixel's RGB values are replaced with the RGB values of its assigned centroid.
6. **Texture Upload:** `SDL_UpdateTexture()` uploads the modified pixel buffer to a streaming GPU texture.
7. **Rendering:** The quantized camera view, 3D RGB cube, K-value label, and palette bar are all rendered to the screen.
8. **Presentation:** `SDL_RenderPresent()` swaps the framebuffer to display.

3.4 Algorithm — K-Means++

The clustering engine is implemented in `kmeans.h` as the function `kmeans2()`. It uses the K-Means++ initialization strategy followed by the standard Lloyd's K-Means iteration. The algorithm processes N pixels (data points), each with D=3 dimensions (R, G, B), and groups them into K clusters.

3.5 OpenMP Parallelization Strategy

OpenMP (Open Multi-Processing) is an API specification for shared-memory parallel programming in C/C++/Fortran. It allows developers to annotate loops and regions for parallel execution using compiler directives (`#pragma omp`). The system uses OpenMP in four critical areas:

3.5.1 Parallel Distance Computation

The assignment step — computing distances from each pixel to each centroid — is parallelized using `#pragma omp parallel for private(j, d) schedule(static)`. The `private(j, d)` clause ensures each thread has its own loop variable copy, and `schedule(static)` pre-divides the N iterations evenly across threads for cache-friendly access patterns. This is the most computationally expensive phase and benefits most from parallelism.

3.5.2 Thread-Private Accumulation

The centroid update step requires accumulating pixel values per cluster. Direct use of shared arrays would cause data races. The solution uses thread-private local arrays (`localCounter`, `localSum`) that are allocated per thread within an `#pragma omp parallel` block, then combined using an `#pragma omp critical` section. This minimizes lock contention while avoiding race conditions.

3.5.3 Parallel Convergence Check

The convergence check compares new and old centroids. This comparison is parallelized with `#pragma omp parallel for collapse(2)` (which fuses the K×D nested loop), and atomic write (`#pragma omp atomic write`) is used to safely update the convergence flag from multiple threads.

3.5.4 K-Means++ Init Parallelism

The initial `minDistSq` computation and updates are parallelized with `schedule(static)` parallel loops. The total sum accumulation uses `#pragma omp parallel for reduction(+:total)` for safe parallel summation.

3.6 Visualization Components

3.6.1 Quantized Camera Feed

The main visual output is the quantized camera image displayed in a resizable viewport (default: 420×240 pixels, positioned at top-left). After clustering, each pixel is assigned the RGB value of its nearest centroid. The modified pixel buffer is uploaded to an SDL3 streaming texture and rendered to the viewport. The visual effect resembles "posterization" — real-world

CHAPTER 4: RESULT ANALYSIS

4.1 Performance Evaluation

The system was tested on a Windows desktop machine with a quad-core Intel CPU at 3.2 GHz, 8 GB RAM, and a standard USB webcam delivering frames at 30 FPS. The camera viewport was set to 420×240 pixels (N = 100,800 pixels). Performance was measured across varying values of K using the system clock (`clock()`) to time the K-Means function.

With K-Means++ initialization, the average number of convergence iterations observed was 3–5 per frame. This is significantly lower than the 15–25 iterations typically observed with random initialization under the same conditions, confirming the theoretical speedup advantage of K-Means++.

4.2 Effect of K on Output Quality and Performance

Table 4.1 summarizes the observed frame rates and visual output quality for different K values at the test viewport resolution (420×240).

K (Clusters)	Approx. Frame Rate	Avg. Iterations	Visual Quality	CPU Load
K = 2	> 60 FPS	2–3	Binary segmentation, high contrast	Very Low
K = 4	~45 FPS	3–4	Good color grouping, recognizable scene	Low
K = 8	~28 FPS	4–5	High quality, most scenes clear	Moderate
K = 12	~15 FPS	5–7	Near-original visual fidelity	High
K = 16	~9 FPS	6–8	Very high, nearly indistinguishable	Very High

Table 4.1: Effect of K on Performance and Visual Output Quality

It is observed that K = 4 to K = 8 provides the best balance between visual quality and real-time performance for the tested hardware configuration.

4.3 Sequential vs. Parallel Execution Time (ms per frame)

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project successfully demonstrates that real-time color quantization of live webcam video frames is achievable using a C-based implementation of the K-Means++ algorithm with OpenMP multi-threading and the SDL3 multimedia framework. The following conclusions are drawn from the implementation and experiments:

1. **K-Means++ Initialization is Effective:** The probabilistic, distance-weighted centroid initialization of K-Means++ consistently reduces the number of iterations required for convergence from an average of 15–25 (random initialization) to 3–5 per frame, providing a 4–6× reduction in iteration count and directly translating to improved real-time performance.
2. **OpenMP Delivers Near-Linear Speedup:** The parallelization of the assignment and update steps using OpenMP achieves approximately 3.4× speedup on a 4-core CPU, consistent with Amdahl's Law for high-parallelizability workloads. This speedup is essential for maintaining real-time frame rates (≥30 FPS) at K values of practical interest.
3. **SDL3 Integration is Efficient:** The use of SDL3's streaming texture API, camera subsystem, and event-driven callback architecture provides a clean, low-overhead framework for building the real-time pipeline with minimal copying of pixel data between CPU and GPU.
4. **Interactive Visualization Enhances Understanding:** The 3D RGB cube plot, luminance-sorted palette bar, and live quantized video feed together provide a comprehensive and intuitive visualization of the clustering process, making this an effective tool for understanding K-Means behavior.
5. **Optimal K Range:** For the tested hardware and resolution, K values between 4 and 8 provide the best trade-off between visual quality and real-time performance.

In summary, this project demonstrates that classical machine learning algorithms, when combined with modern parallel programming techniques and efficient multimedia APIs, can be applied to real-time multimedia pipelines with practical and visually compelling results.

5.2 Future Work

The following enhancements are proposed for future development of this

REFERENCES

[1] D. Arthur and S. Vassilvitskii, "k-means++: The Advantages of Careful Seeding," in *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, New Orleans, Louisiana, 2007, pp. 1027–1035.

[2] S. P. Lloyd, "Least Squares Quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, March 1982. DOI: 10.1109/TIT.1982.1056489.

[3] SDL Development Team, "Simple DirectMedia Layer 3 (SDL3) — Official Documentation and API Reference," 2024. [Online]. Available: https://wiki.libsdl.org/SDL3/FrontPage. [Accessed: April 2026].

[4] OpenMP Architecture Review Board, "OpenMP Application Programming Interface Specification, Version 5.2," November 2021. [Online]. Available: https://www.openmp.org/specifications/. [Accessed: April 2026].

[5] M. Gkotsis, "k-means: A C implementation of the K-Means clustering algorithm," GitHub Repository, 2020. [Online]. Available: https://github.com/Michael-Gkotsis/k-means. [Accessed: April 2026].

[6] R. Reusser, "Webcam K-Means — Real-time Color Quantization Visualization," Interactive Web Demo, 2016. [Online]. Available: https://rreusser.github.io/webcam-kmeans/. [Accessed: April 2026].

[7] P. Heckbert, "Color Image Quantization for Frame Buffer Display," *ACM SIGGRAPH Computer Graphics*, vol. 16, no. 3, pp. 297–307, July 1982. DOI: 10.1145/965145.801294.

[8] B. Chapman, G. Jost, and R. Van Der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*, MIT Press Scientific and Engineering Computation Series. Cambridge, MA: MIT Press, 2008. ISBN: 978-0-262-53302-7.

[9] D. A. Forsyth and J. Ponce, *Computer Vision: A Modern Approach*, 2nd ed. Upper Saddle River, NJ: Pearson Education, 2011. ISBN: 978-0-13-608592-8.

[10] International Telecommunication Union, "ITU-R BT.601-7: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-screen 16:9 Aspect Ratios," ITU, Geneva, Switzerland, 2011.

[11] J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," in *Proceedings of the Fifth Berkeley*